Variance Estimation in `dsRctFedClient`

Continuous vs. Binary Outcomes, Pooled vs. Arm-Specific σ^2

dsRctFedClient package notes

Contents

1 Setup and notation **1**

2 The federated ATE estimator **1**

3 Delta-method variance: unified formula **1**

4 Continuous outcomes **2**
4.1 Gradient vectors 2
4.2 The covariance matrix V_w 2
4.3 Estimating σ^2 via pooled residual sums 2
4.4 Comparison with arm-specific σ^2 3

5 Binary outcomes **3**
5.1 Gradient vectors 3
5.2 The covariance matrix V_w and why there is no σ^2 3

6 Summary **3**

1 Setup and notation

We observe $n = \sum_{k=1}^{K} n_k$ units split across K sites. At each site k, unit i has covariates $x_i \in \mathbb{R}^d$, a binary treatment indicator $W_i \in \{0, 1\}$, and an outcome Y_i.

We fit two arm-specific models (one per treatment value) using federated GLM, broadcasting the pooled coefficients $(\hat{\alpha}_1, \hat{\beta}_1)$ (treated) and $(\hat{\alpha}_0, \hat{\beta}_0)$ (control) to all sites. Write $\tilde{x}_i = (1, x_i^\top)^\top \in \mathbb{R}^{d+1}$ for the augmented covariate vector.

2 The federated ATE estimator

The global ATE estimator is the sample-size-weighted average of local estimates:

$$\hat{\tau} = \frac{1}{n} \sum_{k=1}^{K} n_k \hat{\tau}_k. \tag{1}$$

Continuous outcome (OLS). Each arm is fitted with a linear model $Y_i = \alpha_w + x_i^\top \beta_w + \varepsilon_i$. The local ATE is

$$\hat{\tau}_k = (\hat{\alpha}_1 - \hat{\alpha}_0) + \bar{x}_k^\top (\hat{\beta}_1 - \hat{\beta}_0), \tag{2}$$

where $\bar{x}_k = n_k^{-1} \sum_{i \in k} x_i$ is the local covariate mean.

Binary outcome (logistic, G-computation). Each arm is fitted with a logistic model $\mu_w(x_i) = \sigma(\hat{\alpha}_w + x_i^\top \hat{\beta}_w)$, where $\sigma(\cdot)$ is the logistic function. The local ATE is

$$\hat{\tau}_k = \frac{1}{n_k} \sum_{i \in k} \big[\mu_1(x_i) - \mu_0(x_i)\big]. \tag{3}$$

3 Delta-method variance: unified formula

By the delta method applied to the mapping $(\hat{\alpha}_1, \hat{\beta}_1, \hat{\alpha}_0, \hat{\beta}_0) \mapsto \hat{\tau}$, the asymptotic variance of $\hat{\tau}$ decomposes into a *parametric* term (uncertainty in the GLM coefficients) and a *sampling* term (finite-sample variability of the local ATEs):

$$\mathrm{Var}(\hat{\tau}) = \underbrace{g_1^\top V_1\, g_1 + g_0^\top V_0\, g_0}_{\text{parametric}} + \underbrace{\frac{1}{n(n-1)} \sum_{k=1}^{K} \Big[s_k + n_k \big(\hat{\tau}_k - \hat{\tau}\big)^2 \Big]}_{\text{sampling}}, \tag{4}$$

where:

- $V_w \in \mathbb{R}^{(d+1)\times(d+1)}$ is the covariance matrix of $(\hat{\alpha}_w, \hat{\beta}_w^\top)^\top$,
- $g_1, g_0 \in \mathbb{R}^{d+1}$ are gradient vectors (defined per outcome type in Sections 4 and 5),
- $s_k = \sum_{i \in k} (\delta_i - \hat{\tau}_k)^2$ with $\delta_i = \hat{\mu}_1(x_i) - \hat{\mu}_0(x_i)$ is the local scatter of individual treatment effect estimates around the local mean.

This single formula covers both outcome types. The differences between continuous and binary lie entirely in how g_w and V_w are defined.

4 Continuous outcomes

4.1 Gradient vectors

Under the linear model, $\delta_i = \tilde{x}_i^\top (\hat{\alpha}_1 - \hat{\alpha}_0, \hat{\beta}_1^\top - \hat{\beta}_0^\top)^\top$ is linear in the coefficients, so the gradient of $\hat{\tau}$ with respect to $(\hat{\alpha}_1, \hat{\beta}_1^\top)^\top$ evaluated at the pooled fit is

$$g_1 = \frac{1}{n} \sum_{k=1}^{K} \sum_{i \in k} \tilde{x}_i = \big(1,\, \bar{x}^\top\big)^\top =: \bar{\tilde{x}}, \tag{5}$$

and $g_0 = -\bar{\tilde{x}}$ (same magnitude, opposite sign because increasing the control-arm intercept decreases the ATE).

In practice $\bar{\tilde{x}}$ is assembled from per-site raw sums returned by `localAteDS`: $g_{1,\mathrm{raw},k} = g_{0,\mathrm{raw},k} = \sum_{i \in k} \tilde{x}_i$, and the client forms $g_1 = \sum_k g_{1,\mathrm{raw},k}/n$.

4.2 The covariance matrix V_w

For OLS the coefficient covariance is

$$V_w = \sigma^2 \big(X_w^\top X_w\big)^{-1}, \tag{6}$$

where X_w is the $n_w \times (d+1)$ design matrix of the w-arm and $\sigma^2 = \mathrm{Var}(\varepsilon_i)$ is the noise variance, assumed the same across both arms and all sites.

4.3 Estimating σ^2 via pooled residual sums

`ds.glm` with `viewVarCov = TRUE` returns the *unscaled* matrix $(X_w^\top X_w)^{-1}$, not the full covariance (6). We therefore estimate σ^2 separately from the residual sums of squares collected from all sites and both arms:

$$\hat{\sigma}^2 = \frac{\sum_{k=1}^{K} \mathrm{RSS}_k^{(1)} + \mathrm{RSS}_k^{(0)}}{n - 2(d+1)}, \tag{7}$$

where $\mathrm{RSS}_k^{(w)} = \sum_{i \in k,\, W_i = w} (Y_i - \hat{\alpha}_w - x_i^\top \hat{\beta}_w)^2$. The denominator accounts for $2(d+1)$ estimated parameters (intercept plus d slopes for each of the two arms).

The full covariance used in (4) is then $V_w = \hat{\sigma}^2 (X_w^\top X_w)^{-1}$, with $(X_w^\top X_w)^{-1}$ taken from `ds.glm$VarCovMatrix`.

4.4 Comparison with arm-specific σ^2

An alternative is to use the arm-specific estimate $\hat{\sigma}_w^2 = \mathrm{RSS}^{(w)}/(n_w - (d+1))$, which is what standard `glm` embeds in `vcov(fit)`. The pooled estimator (7) is preferable when:

1. **Homoscedasticity across arms** holds (single σ^2 for both arms). In an RCT this is the natural assumption when treatment does not affect the error distribution.
2. **Efficiency**: the pooled estimate has $n - 2(d+1)$ degrees of freedom versus $n_w - (d+1)$ for each arm separately, halving the estimation noise when arms are balanced.

If treatment shifts the variance of Y, arm-specific estimates are more robust, but this situation lies outside the standard RCT model assumed throughout.

5 Binary outcomes

5.1 Gradient vectors

Under the logistic model, $\delta_i = \mu_1(x_i) - \mu_0(x_i)$ is non-linear in the coefficients. The gradient of $\hat{\tau}$ with respect to $(\hat{\alpha}_1, \hat{\beta}_1^\top)^\top$ is

$$g_1 = \frac{1}{n} \sum_{k=1}^{K} \sum_{i \in k} v_1(x_i)\, \tilde{x}_i, \qquad v_w(x_i) = \mu_w(x_i)\big[1 - \mu_w(x_i)\big], \tag{8}$$

where $v_w(x_i)$ is the logistic variance weight (the derivative of σ). Analogously, $g_0 = -n^{-1} \sum_{k,i} v_0(x_i)\, \tilde{x}_i$.

In practice `localAteDS` returns per-site raw sums $g_{1,\mathrm{raw},k} = \sum_{i \in k} v_1(x_i) \tilde{x}_i$ and $g_{0,\mathrm{raw},k} = \sum_{i \in k} v_0(x_i) \tilde{x}_i$; the client assembles $g_1 = \sum_k g_{1,\mathrm{raw},k}/n$ (and similarly g_0 with a sign flip).

5.2 The covariance matrix V_w and why there is no σ^2

For logistic regression the observed Fisher information is

$$H_w = X_w^\top \operatorname{diag}\big(v_w(x_i)\big)\, X_w, \tag{9}$$

and the maximum-likelihood covariance of $(\hat{\alpha}_w, \hat{\beta}_w^\top)^\top$ is

$$V_w = H_w^{-1}. \tag{10}$$

The key difference from OLS is that H_w already encodes the observation-level variance $\mathrm{Var}(Y_i \mid x_i) = \mu_w(x_i)(1 - \mu_w(x_i))$; there is no separate scale parameter σ^2 to estimate. Formally, the binomial family has *dispersion fixed at 1*.

This is why `ds.glm(viewVarCov = TRUE)$VarCovMatrix` for a binomial model returns the *scaled* covariance H_w^{-1} directly (dispersion = 1, so no scaling is needed), whereas for Gaussian it returns the unscaled $(X_w^\top X_w)^{-1}$. In the implementation this is handled uniformly by setting $\hat{\sigma}^2 = 1$ for binary and multiplying:

$$V_w = \hat{\sigma}^2 \times \texttt{VarCovMatrix} = \begin{cases} \hat{\sigma}^2 (X_w^\top X_w)^{-1} & \text{continuous,} \\ 1 \cdot H_w^{-1} & \text{binary.} \end{cases} \tag{11}$$

6 Summary

	Continuous (OLS)	**Binary (logistic)**
Gradient g_w	$\pm\bar{\tilde{x}} = \pm(1, \bar{x}^\top)^\top$	$\pm n^{-1} \sum_i v_w(x_i)\tilde{x}_i$
`VarCovMatrix` returned by `ds.glm`	$(X_w^\top X_w)^{-1}$ (unscaled)	H_w^{-1} (full covariance)
Scale factor $\hat{\sigma}^2$	$\sum_k (\mathrm{RSS}_k^{(1)} + \mathrm{RSS}_k^{(0)})/(n - 2(d{+}1))$	1 (fixed)
V_w used in (4)	$\hat{\sigma}^2 (X_w^\top X_w)^{-1}$	H_w^{-1}
Extra server disclosure	RSS_k per site, per arm	$\sum_i v_w(x_i)\tilde{x}_i$ per site, per arm

Table 1: Comparison of continuous and binary variance estimation paths.

Both paths feed into the single formula (4), which is implemented in `.compute_variance_delta()` on the client side.